

近端梯度法

[近端梯度法.pdf](#)

从“普通梯度下降处理不了不可导结构项”出发，引入近端算子，再得到近端梯度法，最后用 FISTA 做加速。核心问题是处理：

$$\min_x F(x) = f(x) + g(x),$$

其中 f 是光滑可导的损失项， g 是可能不可导但有结构意义的正则项或约束项。

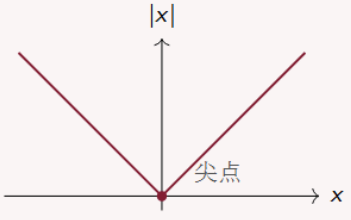
为什么要用近端梯度呢：

在这一章中，我们考虑的函数可以被分为两部分，一部分是方便求导的函数 $f(x)$ ，而另一部分可能是不可导的 $g(x)$ ，但是我们也不上太高的难度，另一部分虽然可能不可导，但是也具有相对简单的结构，比如是正则项，或者是约束指示函数。一个简单的不可导的例子：

例子：绝对值

$$g(x) = |x|$$

在 $x = 0$ 处有尖点，不可导。



但 g 往往有重要作用：例如产生稀疏性、表达约束、编码先验结构。

对于这种结构的函数，我们采取这样的处理方法→对光滑部分做梯度下降，对结构部分用近端算子来修正。（后面会讲到 FISTA，这是一种加速方法）

课件 P6 也给出了我们一个经典例子，也就是 *Lasso* 问题，这个问题可以拆为一个二次项部分和一个一范数，分别对应的就是 $f(x)$ 和 $g(x)$ 。

如何拆为 $f + g$ ？

应用问题	$f(x)$: 光滑损失	$g(x)$: 结构项	结构含义
多指标预测销售额	预测误差	变量选择，希望模型简单	稀疏性
带非加权重的模型	损失函数	权重不能为负	非负约束
图片去噪	与观测图片的差距	边缘清晰、区域平滑	平滑/总变差结构
推荐系统补全评分矩阵	已知评分的拟合误差	希望矩阵低秩	低秩结构

凡是衡量“预测得准不准、拟合得好不好”的，一般放进 f ；凡是表达“解应该长成什么样”的，一般放进 g 。

更标准的数学形式：

问题	$f(x)$	$g(x)$	对应结构
Lasso	二次项	一范数	稀疏、变量选择
约束优化	光滑目标函数	$\delta_{C(x)}$	可行域约束
稀疏逻辑回归	逻辑损失	ℓ_1 正则	稀疏性
图像去噪	数据拟合项	总变差或稀疏正则	平滑、边缘保留
推荐系统	已知评分拟合项	低秩正则或核范数	低秩矩阵结构

其中约束优化可以改写成：

$$\min_{x \in C} f(x) \iff \min_x f(x) + \delta_C(x),$$

$$\delta_C(x) = \begin{cases} 0, & x \in C, \\ +\infty, & x \notin C. \end{cases}$$

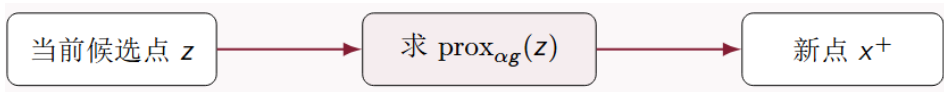
也就是说，约束本身也可以看成一种结构项 $g(x)$

问题结构	是否适合近端梯度
光滑无约束	可用 GD、Newton、BFGS，不一定需要 prox
光滑项 + ℓ_1 正则	非常适合，prox 是软阈值
光滑项 + 简单约束	适合，prox 是投影
复杂不可导项	取决于 prox 是否容易计算

近端算子：

$$\text{prox}_{\alpha g}(z) = \arg \min_x \left\{ g(x) + \frac{1}{2\alpha} \|x - z\|^2 \right\}.$$

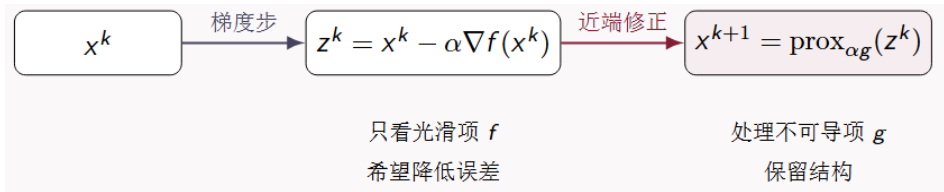
prox 是 proximal 的缩写，可以理解成“靠近的、邻近的”。所以“近端算子”的名字来自它的核心思想：**找一个既靠近 z ，又符合 g 所要求结构的新点。**



所以 *prox* 的本质是：

在靠近 z 的前提下，找一个更符合 g 所偏好结构的点。

这里的 α 控制“允许离 z 多远”。如果 α 很小，那么 $\frac{1}{2\alpha}$ 很大，远离 z 的代价很高。因此 *prox* 会比较保守，新点 x 会更靠近 z 。如果 α 很大，那么 $\frac{1}{2\alpha}$ 比较小，远离 z 的代价变低。因此 *prox* 更愿意按照 $g(x)$ 的结构要求去修正。本质还是惩罚项啦。



处理过程

先像 GD 一样走到 z^k ，然后用 *prox* 把 z^k 修正成满足结构要求的新点。

Proximal Gradient Method

1. 选初始点 x^0 ，步长 $\alpha > 0$ ；
2. 计算光滑项梯度 $g^k = \nabla f(x^k)$ ；
3. 做梯度步 $z^k = x^k - \alpha g^k$ ；
4. 做近端修正 $x^{k+1} = \text{prox}_{\alpha g}(z^k)$ ；
5. 若变化很小或达到最大迭代次数，则停止。

常用 *prox* 典例——软阈值：

软阈值常常用在保证解的稀疏性里，因为通过这个方法解出来的解，那些小的系数会直接被压为 0。通过这个例子，我们把抽象的近端算子具体化。近端算子如果只停留在：

$$\text{prox}_{\alpha g}(z) = \arg \min_x \left\{ g(x) + \frac{1}{2\alpha} \|x - z\|^2 \right\}$$

这就会很抽象；我们来看具体的， ℓ_1 正则的 $prox$ ，明白**很多不可导项虽然不能求梯度，但 $prox$ 可以有非常明确、非常好算的形式。**

ℓ_1 正则的 $prox$ 正是 **软阈值 $S_{\alpha\lambda}(z_i)$** ，它会把小系数直接压成 0，因此能解释 *Lasso* 的变量选择机制。接下来，我们来看**软阈值**究竟是什么：

当：

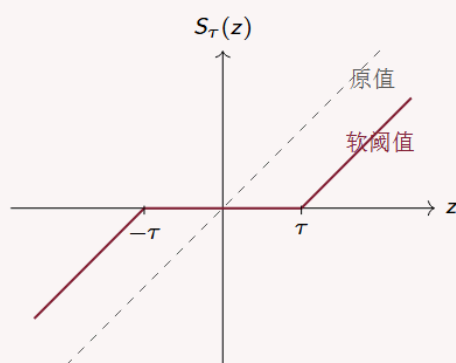
$$g(x) = \lambda \|x\|_1$$

时，有 $\text{prox}_{\alpha\lambda\|\cdot\|_1}(z)_i = S_{\alpha\lambda}(z_i)$ ，**软阈值公式是：**

$$S_\tau(z) = \text{sign}(z) \max\{|z| - \tau, 0\}.$$

它的效果是 $|z| \leq \tau \Rightarrow S_\tau(z) = 0$ 。从**图像**上看就是：

会把靠近 0 的数直接压成 0。



- ▶ $z > \tau$: 向左减去 τ ;
- ▶ $|z| \leq \tau$: 变成 0;
- ▶ $z < -\tau$: 向右加上 τ 。

直觉

小信号被视为噪声或不重要变量，直接清零。

软阈值的来源就是这个优化问题：

$$\min_x \left\{ \lambda \|x\|_1 + \frac{1}{2\alpha} \|x - z\|^2 \right\}$$

因为 $\|x\|_1 = \sum_i |x_i|$, $\|x - z\|^2 = \sum_i (x_i - z_i)^2$ ，所以目标函数可以拆成：

$$\lambda \sum_i |x_i| + \frac{1}{2\alpha} \sum_i (x_i - z_i)^2 = \sum_i \left[\lambda |x_i| + \frac{1}{2\alpha} (x_i - z_i)^2 \right].$$

每个坐标之间互不影响，所以可以**逐坐标求解**。于是我们只需要研究一维问题：

$$\min_x \left\{ \lambda|x| + \frac{1}{2\alpha}(x-z)^2 \right\}$$

把整个目标函数乘以 α ，不改变最小点：

$$\min_x \left\{ \alpha\lambda|x| + \frac{1}{2}(x-z)^2 \right\}$$

记 $\tau = \alpha\lambda$ 于是问题变成：

$$\boxed{\min_x \left\{ \tau|x| + \frac{1}{2}(x-z)^2 \right\}}$$

这就是软阈值真正的来源。因为 $|x|$ 在 $x=0$ 处不可导，所以不能直接全局求导。最自然的做法是按 $x > 0$ 、 $x < 0$ 、 $x = 0$ 分情况。目标函数记作：

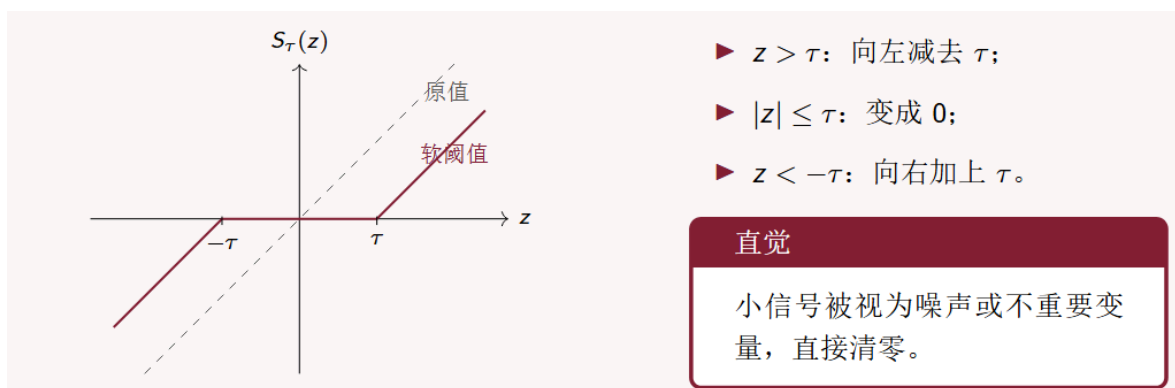
$$\phi(x) = \tau|x| + \frac{1}{2}(x-z)^2$$

情况	目标函数形式	一阶条件	解的成立条件	最优解
$x > 0$	$\tau x + \frac{1}{2}(x-z)^2$	$\tau + x - z = 0$	$x = z - \tau > 0$, 即 $z > \tau$	$x^* = z - \tau$
$x < 0$	$-\tau x + \frac{1}{2}(x-z)^2$	$-\tau + x - z = 0$	$x = z + \tau < 0$, 即 $z < -\tau$	$x^* = z + \tau$
$x = 0$	不可导点	左右导数跨过 0	$-\tau \leq z \leq \tau$	0

因此合并得到：

$$S_\tau(z) = \begin{cases} z - \tau, & z > \tau, \\ 0, & |z| \leq \tau, \\ z + \tau, & z < -\tau. \end{cases}$$

$$S_\tau(z) = \text{sign}(z) \max\{|z| - \tau, 0\}.$$



假如我们设阈值为 0.4，那么对于向量 $z = (1.2, 0.3, -0.2, -1.5, 0.8)$ ，我们逐个做软阈值操作，则有新的向量就变成了 $z^* = (0.8, 0, 0, -1.1, 0.4)$ 。

所以一句话来总结，在 ℓ_1 正则情况下的近端梯度，我们的处理方式就是先用普通梯度步，得到临时向量 z ，再用软阈值逐个处理，小的清零大的向零收缩。

近端梯度法的算法框架：

可以整理成下面这张表（我们重点掌握前三类，也就是软阈值、平方缩放、以及约束指示函数对应投影）：

$g(x)$ 类型	prox 做什么	结构效果
$\lambda x _1$	逐坐标软阈值	稀疏性
$\frac{\lambda}{2} x _2^2$	整体按比例缩小	系数变小，不清零
$\delta_{\{x \geq 0\}}(x)$	负数截成 0	非负约束
$\delta_{[l, u]}(x)$	截断到区间 $[l, u]$	盒约束
$\lambda x _2$	向量整体软阈值	整体收缩/组选择
$\lambda \sum_G x_G _2$	每组整体软阈值	Group Lasso
$\delta_C(x)$	投影到 C	一般约束
$\lambda X _*$	对奇异值软阈值	低秩矩阵

我们来解释一下软阈值、平方缩放、以及约束指示函数对应投影又是怎么推导出来的，首先这三类操作都不是一拍脑子决定的，而是从同一个定义里解出来的，一切又回到了我们近端修正的公式：

$$\text{prox}_{\alpha g}(z) = \arg \min_x \left\{ g(x) + \frac{1}{2\alpha} \|x - z\|^2 \right\}$$

平方缩放：

当近端算子为下面这个式子时：

$$\text{prox}_{\alpha g}(z) = \arg \min_x \left\{ \frac{\lambda}{2} \|x\|_2^2 + \frac{1}{2\alpha} \|x - z\|^2 \right\}.$$

这个问题是光滑的，可以**直接求导**。目标函数为：

$$\phi(x) = \frac{\lambda}{2} \|x\|_2^2 + \frac{1}{2\alpha} \|x - z\|^2.$$

对 x 求梯度有 $\nabla \phi(x) = \lambda x + \frac{1}{\alpha}(x - z)$ ，令梯度为 0：

$$\lambda x + \frac{1}{\alpha}(x - z) = 0.$$

整理有 $(1 + \alpha\lambda)x = z$ ，所以：

$$x^* = \frac{1}{1 + \alpha\lambda} z.$$

它的操作逻辑是：把整个向量按同一个比例缩小。 比如：

$$z = (2, -4, 6), \quad \alpha\lambda = 1,$$

那么就有 $\text{prox}(z) = \frac{1}{2}z = (1, -2, 3)$ ，它和软阈值最大的区别是，**软阈值会把某些坐标清零，而平方缩放一般只是压缩大小**。所以平方 ℓ_2 正则类似 ridge 正则：它让系数更小、更平滑，但不做变量选择。

约束指示函数对应投影：

令 $g(x) = \delta_C(x)$ ，其中：

$$\delta_C(x) = \begin{cases} 0, & x \in C, \\ +\infty, & x \notin C. \end{cases}$$

该函数的作用是强制 x 必须在集合 C 里面。代入 $prox$ 定义：

$$\text{prox}_{\alpha\delta_C}(z) = \arg \min_x \left\{ \delta_C(x) + \frac{1}{2\alpha} \|x - z\|^2 \right\}.$$

现在分两类点看。如果 $x \notin C$ ，那么 $\delta_C(x) = +\infty$ ，这个点不可能成为最优解。如果 $x \in C$ ，那么 $\delta_C(x) = 0$ 。目标函数就只剩下：

$$\frac{1}{2\alpha} \|x - z\|^2$$

所以问题变成 $\arg \min_{x \in C} \|x - z\|^2$ ，这正是投影的定义 $\Pi_C(z) = \arg \min_{x \in C} \|x - z\|^2$ ，因此：

$$\text{prox}_{\alpha\delta_C}(z) = \Pi_C(z).$$

也就是说，约束指示函数的 $prox$ 就是把点拉回可行域。

两个具体的投影例子（这个知识点在前面的投影梯度法那一章节有更详细的讲解）：

非负约束投影：

如果 $C = \{x : x \geq 0\}$ ，那么：

$$\text{prox}_{\alpha\delta_C}(z) = \Pi_C(z) = \max\{z, 0\}.$$

这是逐坐标操作： $z_i < 0 \Rightarrow x_i = 0$ ， $z_i \geq 0 \Rightarrow x_i = z_i$ ，例如：

$$z = (2, -3, 0.5, -0.1),$$

投影后： $\max\{z, 0\} = (2, 0, 0.5, 0)$

盒约束投影：

如果 $C = [l, u]$ ，即每个坐标要满足 $l \leq x_i \leq u$ ，那么投影为：

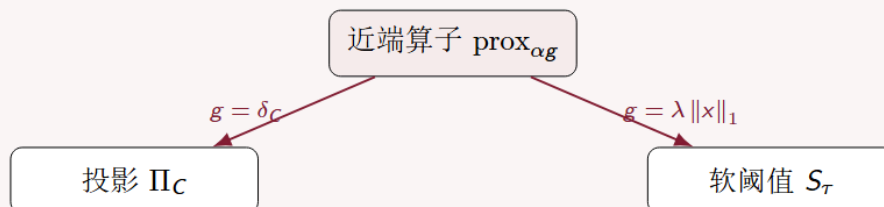
$$\Pi_C(z) = \min\{u, \max\{l, z\}\}.$$

它的含义是： $z_i < l \Rightarrow x_i = l$ ， $l \leq z_i \leq u \Rightarrow x_i = z_i$ ， $z_i > u \Rightarrow x_i = u$ 。比如：

$$l = 0, \quad u = 1, \\ z = (-0.3, 0.2, 0.8, 1.5),$$

则投影后就为： $\Pi_{[0,1]}(z) = (0, 0.2, 0.8, 1)$

投影和软阈值看起来不同，但都属于 prox。



统一理解

近端算子根据 g 的类型做不同修正：约束对应投影， ℓ_1 正则对应软阈值。

ISTA——Lasso 的近端梯度法：

ISTA 是 **Iterative Shrinkage-Thresholding Algorithm**，中文可以理解为：迭代收缩阈值算法。实际上就是 Lasso 这个具体问题下的近端梯度算法。Lasso 问题是：

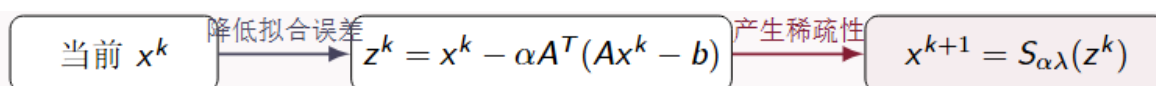
$$\min_x \frac{1}{2} \|Ax - b\|^2 + \lambda \|x\|_1.$$

按照近端梯度法的思路，把它拆成： $f(x) = \frac{1}{2} \|Ax - b\|^2$ ， $g(x) = \lambda \|x\|_1$ ，其中 $f(x)$ 是光滑的平方损失，负责拟合数据，而 $g(x)$ 是不可导的 ℓ_1 正则项，负责产生稀疏性。

Lasso 的核心公式：

$$x^{k+1} = S_{\alpha\lambda} (x^k - \alpha A^T (Ax^k - b))$$

里面这一坨： $x^k - \alpha A^T (Ax^k - b)$ 是**梯度下降步**，处理平方损失。外面这个： $S_{\alpha\lambda}(\cdot)$ 是**软阈值操作**，用来处理 ℓ_1 正则。



一步 ISTA 就可以写成： $x^k \longrightarrow z^k = x^k - \alpha A^T (Ax^k - b) \longrightarrow x^{k+1} = S_{\alpha\lambda}(z^k)$

近端梯度法的停止准则：

和我们之前说的投影梯度法的停止准则定义非常相似，我们这里也定义：

$$G_{\alpha}(x) = \frac{1}{\alpha} [x - \text{prox}_{\alpha g}(x - \alpha \nabla f(x))]$$

它叫**近端梯度映射**。本质上在衡量的是，如果再做一步近端梯度更新，当前点会不会有明显的变化。当没有结构项时，也就是 $g(x) = 0$ 时，于是： $G_{\alpha}(x) = \frac{1}{\alpha} [x - (x - \alpha \nabla f(x))] = \nabla f(x)$ ，所以在普通光滑优化里，近端梯度映射就退化成普通梯度。说明近端梯度法是带结构约束的梯度下降法。

步长与收敛速度：

近端梯度法里面，梯度步是： $x - \alpha \nabla f(x)$ ，如果 α 太大，梯度步可能一下子冲过头。后面的 prox 虽然会修正结构，但它不能完全弥补过大的梯度步。所以需要：

$$\alpha \leq \frac{1}{L}$$

这和普通梯度下降的逻辑也是一致的：**步长不能超过由光滑性决定的安全范围**。这里 L 是 f 的光滑常数，也就是梯度 Lipschitz 常数。衡量的是 f 的梯度变化有多快。如果 L 很大，说明函数弯得厉害，步长就要小一点；如果 L 较小，函数比较平缓，步长可以大一点。所以我们常用固定步长条件：

$$0 < \alpha \leq \frac{1}{L}$$

Lasso 的光滑部分是： $f(x) = \frac{1}{2} \|Ax - b\|^2$ ，它的梯度是：

$$\nabla f(x) = A^T(Ax - b)$$

这个梯度的 Lipschitz 常数是 $L = \lambda_{\max}(A^T A)$ ，这里 $\lambda_{\max}(A^T A)$ 表示矩阵 $A^T A$ 的最大特征值。所以 Lasso 里常用：

$$0 < \alpha \leq \frac{1}{\lambda_{\max}(A^T A)}$$

在收敛速度上, 如果 f 是凸且 L -光滑, g 是凸函数, 并且: $0 < \alpha \leq \frac{1}{L}$, 那么近端梯度法满足:

$$F(x^k) - F(x^*) = O\left(\frac{1}{k}\right)$$

其中 $F(x) = f(x) + g(x)$, 其实这里也很好理解, **因为近端梯度从本质上就是 GD 面向符合优化的推广**, 所以他的**收敛速度与梯度下降是在一个量级的**。速度仍但是 $O(1/k)$, 但是能处理不可导结构了。

从 ISTA 到 FISTA —— 进一步加速:

ISTA 的优点是简单、稳定、容易实现; 但是它的速度在大型问题中还是偏慢, 并且没有利用到历史信息, 所以 FISTA 算法其实就是看, 能不能像 Nesterov 那样, **在近端梯度法里也利用历史信息进行加速**。ISTA 是:

$$x^{k+1} = \text{prox}_{\alpha g}(x^k - \alpha \nabla f(x^k))$$

也就是直接从当前点 x^k 做近端梯度步。FISTA 不一样。它先构造一个**外推点**:

$$y^k = x^k + \beta_k(x^k - x^{k-1})$$

这里: $x^k - x^{k-1}$, 表示最近一步的移动方向。 β_k 是动量系数, 控制沿着历史方向“多冲一点”的程度。然后 FISTA 不在 x^k 上**做近端梯度步**, 而是在 y^k 上做:

$$x^{k+1} = \text{prox}_{\alpha g}(y^k - \alpha \nabla f(y^k))$$

方法	从哪里出发做近端梯度步	是否利用历史方向
ISTA	当前点 x^k	不利用
FISTA	外推点 y^k	利用 $x^k - x^{k-1}$

直观理解

ISTA 是“从当前点走”; FISTA 是“先根据历史趋势预测一个点, 再从预测点做近端梯度步”。

然而如果方向是对的，FISTA 能帮我们少走弯路，但是方向如果错了，反而帮倒忙，所以 FISTA 尽管**理论上收敛速度能够达到 $O(1/k^2)$** ，**更快，但也更容易会振荡。**

方法	处理问题	核心思想
GD	光滑无约束	沿负梯度走
Nesterov	光滑无约束	在预测点计算梯度
ISTA	光滑项 + 不可导项	梯度步 + prox
FISTA	光滑项 + 不可导项	Nesterov 外推 + prox

FISTA 具体在 Lasso 问题上的应用：

对 Lasso

$$\min_x \frac{1}{2} \|Ax - b\|^2 + \lambda \|x\|_1,$$

FISTA 为

$$\begin{aligned} y^k &= x^k + \beta_k(x^k - x^{k-1}), \\ x^{k+1} &= S_{\alpha\lambda}(y^k - \alpha A^T(Ay^k - b)). \end{aligned}$$

和 ISTA 的差别

ISTA 对 x^k 做软阈值梯度步；FISTA 先生成预测点 y^k ，再对 y^k 做软阈值梯度步。

总而言之，FISTA 可以看作是近端梯度版的 Nesterov 加速。